

한국어-영어 의미 대응 문장쌍의 Tokenization Premium 측정· 분해 및 설명요인 분석

UTF-8 표현, 표면형 구조, 형태소 특성 및 Subword Tokenization

2026-08-16

차례

0. 문서 통제 정보	4
0.1 변경 로그	5
1. 연구명	5
1.1 국문	5
1.2 영문	5
1.3 용어 선택	5
2. Executive Summary	6
3. 연구의 범위와 비범위	6
3.1 In Scope	6
3.2 Out of Scope	6
4. 이론적 배경과 연구 위치	7
5. 핵심 개념과 처리 모형	7
5.1 핵심 구분	8
6. 연구 질문과 사전 가설	8
6.1 RQ1 — 전체 Tokenization Premium	8
6.2 RQ2 — 구조적 분해	8
6.3 RQ3 — 표면형 구조	9
6.4 RQ4 — 형태소학적 특성	9
6.5 RQ5 — Tokenizer 내부 구조와의 연결	9
6.6 RQ6 — 이질성	9
6.7 RQ7 — Serving-level 차이	9
7. 분석 단위와 estimand 체계	9
7.1 기본 분석 단위	9
7.2 주·보조 결과변수	9
8. Exact Decomposition	10
8.1 Tokenization Premium	10
8.2 Code Point Length Ratio	10
8.3 Byte Density Ratio	10
8.4 Tokenizer Compression Penalty	10
9. 데이터 설계	11
9.1 권장 표본 규모	11
9.2 층화 변수	11
9.3 데이터 source 계층	11
10. Pair Quality Control	12
10.1 Hard exclusion	12
10.2 Soft flags	12
10.3 의미 대응 QC	12
11. 정규화 규약	12

11.1 저장 필드	12
11.2 금지되는 무음 정규화	13
12. 데이터 스키마	13
12.1 D-01 Pair Registry	13
12.2 D-02 Representation Features	13
12.3 D-03 Morphology Measurement	14
POS feature mapping 원칙	14
12.4 D-04 Token Measurement	14
12.5 D-05 Regex Chunk Measurement	15
12.6 D-06 Model Run	15
12.7 D-07 Cost / Compute Snapshot	16
13. 핵심 feature 정의	16
13.1 Token Premium	16
13.2 절대 Token 차이	16
13.3 Byte Density	16
13.4 Whitespace Density	16
13.5 Pair Length Ratio	16
13.6 Morpheme Density	17
13.7 Particle / Ending Ratio	17
14. Tokenizer 고정 규약	17
14.1 Primary tokenizer	17
14.2 측정 원칙	17
14.3 o200k_harmony와의 관계	17
15. 형태소 분석 고정 규약	17
15.1 Primary analyzer	17
15.2 분석 금지 사항	18
15.3 형태소 feature block	18
16. 기초 분석 계획	18
16.1 핵심 분포	18
16.2 필수 시각화	18
16.3 Extreme-case audit	19
17. RQ1 통계 추론	19
17.1 Primary test	19
17.2 Confidence intervals	19
17.3 보고 원칙	19
18. 메인 설명모형	19
18.1 Outcome A — Total Premium	19
18.2 Outcome B — Compression Penalty	20
19. 단계적 모형 비교	20
19.1 Model blocks	20
19.2 핵심 질문	20
19.3 M3의 역할	21

20. Source effect와 식별 가능성	21
20.1 Random vs Fixed	21
20.2 Identifiability Gate	21
21. 공선성 및 compositional feature 처리	21
22. 예측모형	21
23. 데이터 분할과 일반화 검증	22
23.1 설명모형	22
23.2 예측모형	22
24. Robustness / Sensitivity Analysis	22
25. Track A — Tokenizer Intrinsic Analysis	22
25.1 목적	22
25.2 입력	22
25.3 산출	23
25.4 성공 조건	23
26. Track B — gpt-oss Serving Experiment	23
26.1 목적	23
26.2 모델	23
26.3 세 가지 protocol	23
B1. Request-only accounting	23
B2. Controlled generation	23
B3. Natural generation	24
26.4 고정 변수	24
26.5 latency 측정	24
27. Serving-level token ratio	24
28. 비용 모형	25
28.1 Provider per-token billing	25
28.2 Local deployment	25
29. 다중 검정과 통계 보고	25
30. 재현성 규약	26
30.1 환경	26
30.2 데이터 lineage	26
30.3 Seed policy	26
31. 연구 Gate	26
G0 — Design Freeze	26
G1 — Data Integrity	27
G2 — Representation Integrity	27
G3 — Tokenizer Integrity	27
G4 — Morphology Integrity	27
G5 — Analysis Readiness	27
G6 — Release	27

32. 핵심 위협과 해석 제한	28
T-03 Translationese	28
T-04 Domain-source confounding	28
T-05 Analyzer measurement error	28
T-06 Deterministic feature overlap	28
T-07 Tokenizer-specific generalization	28
T-08 Cost generalization	28
33. 결과 해석 프레임	28
Case A — TP > 1, ByteDensity가 주로 설명	28
Case B — TP > 1, CompressionPenalty도 큼	28
Case C — Morphology block이 M1 대비 실질적으로 개선	28
Case D — Morphology block 증분이 거의 없음	28
Case E — Domain interaction이 큼	29
34. Traceability Matrix	29
35. 최종 표·그림 설계	29
Tables	29
Figures	29
36. 프로젝트 정보구조(IA)	30
37. Notebook 실행 순서	31
Phase 0 — Environment	31
Phase 1 — Data Contract	31
Phase 2 — Normalization & QC	31
Phase 3 — Feature Layer	31
Phase 4 — Tokenizer Measurement	32
Phase 5 — Statistics	32
Phase 6 — Serving	32
Phase 7 — Release	32
38. 파일 명명 규칙	32
39. 분석 전 체크리스트	33
40. 최종 논문/보고서 서술용 핵심 문단	33
41. 참고문헌 및 구현 기준 자료	34
Appendix A. 결정 로그	34
Appendix B. 최소 산출물 계약	34
0. 문서 통제 정보	

항목	값
문서 ID	KOEN-TP-RS-001
버전	v1.0
상태	FINAL DESIGN / PRE-ANALYSIS

항목	값
기준일	2026-08-16
주 분석 단위	의미적으로 대응되는 한국어-영어 문장쌍
주 tokenizer	o200k_base
주 형태소 분석기	Kiwi / kiwipiepy 계열, 실행 버전 고정
주 결과변수	log_token_premium
추론 성격	paired observational analysis; 인과효과가 아닌 조건부 연관성
연구 트랙	Track A: tokenizer intrinsic / Track B: serving-level auxiliary experiment

문서 역할. 이 문서는 연구 질문, estimand, 데이터 구조, 변수 정의, 전처리, tokenizer 측정, 형태소 feature, 통계모형, 예측모형, serving 실험, 재현성 규약 및 산출물 구조를 하나의 SSOT(Single Source of Truth)로 고정한다. 분석 중 설계 변경이 필요한 경우 본문을 덮어쓰지 않고 변경 로그에 사유와 영향을 기록한다.

0.1 변경 로그

버전	일자	핵심 변경
v1.0	2026-08-16	연구 질문-측정-모형 정합성 재설계, exact decomposition 추가, RQ1 검정대상 수정, tokenizer regex chunking과 형태소 분석 분리, Track B gpt-oss/Harmony 조건 정교화

1. 연구명

1.1 국문

한국어-영어 의미 대응 문장쌍의 Tokenization Premium 측정·분해 및 설명요인 분석: UTF-8 표현, 표면형 구조, 형태소 특성 및 Subword Tokenization

1.2 영문

Measuring, Decomposing, and Explaining the Korean-English Tokenization Premium in Semantically Matched Sentence Pairs: UTF-8 Representation, Surface-Form Structure, Morphological Features, and Subword Tokenization

1.3 용어 선택

본 연구는 Tokenization Premium을 정식 용어로 사용하고, 문맥상 반복을 줄이기 위해 Token Premium 또는 TP를 약칭으로 사용한다. 이는 특정 언어가 동일하거나 대응되는 의미 내용을 표현할 때 기준 언어보다 더 많은 tokenizer token을 요구하는 현상을 가리킨다.

Method Note MN-01 — 제목의 “설명”은 인과를 뜻하지 않는다. 본 연구의 회귀모형은 문자열 구조와 tokenizer 산출량 사이의 조건부 통계적 연관성을 설명한다. 형태소 feature의 회귀계수는 “형태소가 token premium을 원인적으로 발생시킨다”는 효과로 해석하지 않는다.

2. Executive Summary

본 연구는 의미적으로 대응되는 한국어-영어 문장쌍을 대상으로 특정 tokenizer에서 한국어가 영어보다 얼마나 많은 token을 사용하는지 측정하고, 그 차이를 UTF-8 표현 부담, 문장 길이와 공백·문자군 등 표면형 구조, 한국어 형태소학적 특성, 그리고 tokenizer의 구현상 regex chunking 및 subword compression 구조의 관점에서 분석한다.

핵심은 단순한 평균 token 수 비교가 아니다. 본 연구는 Tokenization Premium을 다음의 정확한 곱셈 항등식으로 분해한다.

$$\frac{T_{i,KO}}{T_{i,EN}} = \frac{C_{i,KO}}{C_{i,EN}} \times \frac{B_{i,KO}/C_{i,KO}}{B_{i,EN}/C_{i,EN}} \times \frac{T_{i,KO}/B_{i,KO}}{T_{i,EN}/B_{i,EN}}$$

여기서 C 는 Unicode code point 수, B 는 UTF-8 byte 수, T 는 tokenizer token 수다. 따라서 전체 premium은 다음 세 부분으로 구조적으로 구분된다.

1. **표현 길이 차이**: 동일 의미를 표현하는 데 필요한 code point 수의 차이
2. **UTF-8 byte density 차이**: code point 하나를 UTF-8로 표현하는 데 필요한 평균 byte 수의 차이
3. **tokenizer compression 차이**: byte당 token 수가 언어별로 얼마나 다른지

로그를 취하면 다음의 정확한 가법적 분해가 된다.

$$\log TP_i = \log \text{CodePointRatio}_i + \log \text{ByteDensityRatio}_i + \log \text{CompressionPenalty}_i$$

이 분해를 통해 “한국어가 UTF-8에서 더 많은 byte를 쓰기 때문에 token이 많다”와 “동일한 byte량에서도 tokenizer가 한국어를 더 잘게 나누기 때문에 token이 많다”를 같은 주장으로 취급하지 않는다.

형태소 분석은 tokenizer pipeline 내부 단계가 아니다. 형태소 분석기는 한국어 표면형의 문법적·구조적 특징을 정량화하는 외부 분석기이며, tokenizer의 regex chunk 경계나 최종 BPE token 경계와 일치한다고 가정하지 않는다. 형태소 feature는 표면형·길이·도메인·출처를 통제된 뒤 Tokenization Premium 또는 tokenizer compression penalty에 추가 설명력을 제공하는지 검증하는 데 사용한다.

3. 연구의 범위와 비범위

3.1 In Scope

- 의미 대응 KO-EN pair의 token count 차이와 Tokenization Premium 측정
- UTF-8 byte, code point, grapheme, 공백, 문자군, 숫자·구두점 등 representation feature 추출
- 한국어 형태소 밀도, 조사·어미·접사 관련 feature 추출
- 고정된 o200k_base 구현의 regex chunking과 최종 BPE tokenization 측정
- 도메인·문장 유형·번역 방향·출처별 이질성 분석
- mixed-effects 또는 source fixed-effects 기반 설명모형
- Elastic Net, gradient boosting 등 예측모형을 통한 비선형·interaction 탐색
- gpt-oss 계열을 이용한 serving-level request/output/latency 실험
- 가격 또는 로컬 compute cost를 별도 snapshot으로 기록하는 선택적 비용 분석

3.2 Out of Scope

- 형태소 분석 결과를 tokenizer의 실제 내부 token 경계로 간주하는 주장
- token premium의 원인을 형태론 하나로 귀속하는 인과 주장
- tokenizer 학습 corpus 구성이나 merge history를 완전히 역추정하는 작업

- 모델의 한국어 능력 전반을 token count만으로 평가하는 작업
- 가격 snapshot을 영구적 비용 구조로 일반화하는 주장
- gpt-oss-20b와 120b의 모델 크기 차이가 동일 직렬화 입력의 token count 자체를 바꾼다는 주장

4. 이론적 배경과 연구 위치

다국어 tokenizer 연구는 동일 의미를 표현하는 언어 간 tokenization length가 크게 달라질 수 있으며, 그 차이가 비용, 처리 가능한 유효 context, latency 및 downstream utility와 연결될 수 있음을 보여 왔다. 선행연구는 특히 UTF-8 byte 표현, script, vocabulary allocation, pre-tokenization 및 subword segmentation이 언어별 격차에 관여할 수 있음을 지적한다.

한국어는 교착적 형태론, 어절 단위 공백, 조사·어미 결합, 고유한 Hangul script를 동시에 갖기 때문에 단순히 “문자당 byte가 많다” 또는 “형태소가 많다” 중 하나로 tokenization inefficiency를 설명하기 어렵다. 한국어 전용 tokenization 연구 역시 과분절(over-segmentation)과 형태론을 고려한 vocabulary/tokenization 설계의 필요성을 별도로 다뤘다.

본 연구의 의도된 기여는 새로운 tokenizer를 제안하는 것이 아니라, **의미 대응 KO-EN pair에서 전체 premium을 구조적으로 분해하고, 표면형과 형태소 feature가 어떤 추가 설명력을 갖는지 재현 가능한 measurement design으로 검증하는 것이다.**

Research Position RP-01. 본 연구는 tokenizer fairness 연구와 한국어 형태론 기반 tokenization 연구 사이에 위치한다. 단순 비용 감사(cost audit)보다 미시적 feature 수준의 설명을 강화하되, 형태소 기반 tokenizer를 새로 학습하는 연구와는 구분한다.

5. 핵심 개념과 처리 모형

문장쌍 i , 언어 $l \in \{KO, EN\}$ 에 대해 텍스트의 tokenizer 처리를 다음과 같이 표현한다.

$$x_{i,l} \rightarrow b_{i,l} \rightarrow P_v(b_{i,l}) \rightarrow T_v(P_v(b_{i,l})) \rightarrow N_{i,l,v}$$

- $x_{i,l}$: 정규화된 입력 문자열
- $b_{i,l} = \text{UTF8}(x_{i,l})$: UTF-8 byte sequence
- P_v : 버전 v 에서 관측되는 tokenizer의 1차 regex chunking
- T_v : 동일 버전의 BPE/subword encoding
- $N_{i,l,v}$: 최종 token count

한국어 형태소 분석은 별도 함수다.

$$M_a(x_{i,KO}) = \{(m_{i1}, \tau_{i1}), \dots, (m_{ik}, \tau_{ik})\}$$

- a : 형태소 분석기 및 모델 버전
- m_{ij} : 형태소 표면형 또는 정규화형
- τ_{ij} : 품사/문법 태그
- k : 형태소 수

예를 들어 언어학적으로 다음과 같은 분해가 가능하다.

먹었다 = 먹 + 었 + 다

학생들에게도 = 학생 + 들 + 에게 + 도

그러나 이 경계는 tokenizer의 regex chunk 또는 BPE token 경계와 같을 필요가 없다.

5.1 핵심 구분

구분	형태소 분석	Regex chunking / pre-token stage	최종 subword tokenization
목적	문법·형태 구조 분석	BPE 전 문자열 1차 분할	모델 입력용 token ID 생성
기준	어근, 접사, 조사, 어미, POS	Unicode category, 공백, 숫자, 구두점, regex	mergeable ranks, vocabulary, special token 규칙
출력	morpheme + POS	chunk sequence	token ID sequence
언어학적 경계	중요	보장하지 않음	보장하지 않음
버전 민감도	분석기/모델에 의존	tokenizer 구현에 의존	매우 큼
연구상 역할	외생적 설명 feature	tokenizer mechanism audit	결과변수 생성 과정

Reproducibility Rule RR-01. pre-tokenization이라는 일반명 대신 실제 구현을 다룰 때는 o200k_base regex chunking이라고 기록한다. 구현의 pat_str 자체와 hash를 저장한다.

6. 연구 질문과 사전 가설

6.1 RQ1 — 전체 Tokenization Premium

의미 대응 KO-EN 문장쌍에서 o200k_base 기준 한국어 Tokenization Premium은 1보다 큰가?
주 estimand:

$$\theta_{TP} = \text{Median}(\log TP_i)$$

주 가설:

$$H_0 : \theta_{TP} = 0 \quad H_1 : \theta_{TP} > 0$$

보조 estimand:

$$P(TP_i > 1), \quad \text{Median}(TP_i), \quad \exp(E[\log TP_i])$$

Decision D-01. RQ1의 주 검정은 $T_{KO} - T_{EN}$ 이 아니라 $\log(TP)$ 에 대해 수행한다. 절대 token 차이 ΔT 는 단위가 문장 길이에 의존하므로 보조 결과로 분리한다.

6.2 RQ2 — 구조적 분해

전체 premium 가운데 다음 세 요소가 각각 어느 정도의 분포를 보이는가?

1. CodePointRatio
2. ByteDensityRatio
3. CompressionPenalty

정확한 분해식:

$$TP_i = \text{CodePointRatio}_i \times \text{ByteDensityRatio}_i \times \text{CompressionPenalty}_i$$

6.3 RQ3 — 표면형 구조

공백 밀도, grapheme 구조, Hangul/Latin/digit/punctuation 비율, script mixing, 문장 길이, 숫자·특수기호가 $\log(\text{TP})$ 와 $\log(\text{CompressionPenalty})$ 에 어떤 조건부 연관을 갖는가?

6.4 RQ4 — 형태소학적 특성

형태소 밀도, 조사 비율, 어미 비율, 접사 비율이 길이·byte·공백·문자군·도메인·출처를 통제한 후에도 $\log(\text{TP})$ 또는 $\log(\text{CompressionPenalty})$ 의 추가 설명력을 제공하는가?

주 판단은 개별 p-value보다 **형태소 feature block의 증분 설명력**에 둔다.

6.5 RQ5 — Tokenizer 내부 구조와의 연결

고정된 o200k_base 구현에서 관측되는 regex chunk count, chunk 길이, token-per-chunk, token-per-byte 등의 mechanism feature가 최종 premium과 어떻게 연결되는가?

Method Note MN-02 — post-treatment 성격. tokenizer-derived chunk feature는 최종 token 수의 기계적 중간 단계에 가깝다. 따라서 이를 포함한 모형은 “형태소 효과를 더 엄격히 통제한 주 회귀”가 아니라 **mechanism audit**로 분리한다. M3 결과를 근거로 M2의 형태소 연관성을 인과적으로 제거하거나 확정하지 않는다.

6.6 RQ6 — 이질성

도메인, 문장 유형, 번역 방향, 출처 및 길이 strata에 따라 Tokenization Premium과 설명요인의 크기가 달라지는가?

6.7 RQ7 — Serving-level 차이

동일 의미·동일 형식의 KO/EN 요청을 gpt-oss-20b와 gpt-oss-120b에 입력했을 때 다음이 어떻게 달라지는가?

- serialized request token
- analysis/reasoning token(관측 가능 시)
- final output token
- total token
- time to first token(TTFT)
- end-to-end latency
- decode throughput
- provider cost 또는 local compute proxy

7. 분석 단위와 estimand 체계

7.1 기본 분석 단위

$$(x_{i,\text{KO}}, x_{i,\text{EN}}), \quad i = 1, \dots, N$$

동일 pair 내부에서 언어를 비교하므로 topic과 의도는 pair matching으로 통제한다. 다만 번역문은 완전히 동일한 surface semantics가 아니며 번역 방향·의역·translationese가 남을 수 있으므로 translation_direction과 pair quality를 별도 관리한다.

7.2 주·보조 결과변수

계층	변수	역할
Primary	log_token_premium	RQ1 및 메인 설명모형

계층	변수	역할
Secondary	token_premium	해석 가능한 ratio 보고
Secondary	log_compression_penalty	byte량을 분리한 tokenizer compression 분석
Secondary	token_difference	실제 token 절대량 차이
Descriptive	ko_tokens_per_byte, en_tokens_per_byte	언어별 compression
Descriptive	ko_tokens_per_codepoint, en_tokens_per_codepoint	문자 기준 fertility 유사 지표

8. Exact Decomposition

문장쌍 i 에 대해 다음을 정의한다.

- $T_{i,l}$: token count
- $B_{i,l}$: UTF-8 byte count
- $C_{i,l}$: Unicode code point count

8.1 Tokenization Premium

$$TP_i = \frac{T_{i,KO}}{T_{i,EN}}$$

8.2 Code Point Length Ratio

$$\text{CodePointRatio}_i = \frac{C_{i,KO}}{C_{i,EN}}$$

8.3 Byte Density Ratio

언어별 byte density:

$$D_{i,l}^B = \frac{B_{i,l}}{C_{i,l}}$$

pair ratio:

$$\text{ByteDensityRatio}_i = \frac{D_{i,KO}^B}{D_{i,EN}^B}$$

8.4 Tokenizer Compression Penalty

언어별 token-per-byte:

$$E_{i,l} = \frac{T_{i,l}}{B_{i,l}}$$

pair ratio:

$$\text{CompressionPenalty}_i = \frac{E_{i,\text{KO}}}{E_{i,\text{EN}}}$$

따라서 다음이 정확히 성립한다.

$$TP_i = \text{CodePointRatio}_i \cdot \text{ByteDensityRatio}_i \cdot \text{CompressionPenalty}_i$$

로그 공간에서는:

$$\log TP_i = \log \text{CodePointRatio}_i + \log \text{ByteDensityRatio}_i + \log \text{CompressionPenalty}_i$$

Interpretation Rule IR-01. ByteDensityRatio가 크다는 사실과 CompressionPenalty가 크다는 사실을 분리해 보고한다. 전자는 UTF-8 표현 구조, 후자는 고정 tokenizer가 같은 byte량을 얼마나 잘 압축하는지에 더 직접적으로 대응한다.

9. 데이터 설계

9.1 권장 표본 규모

단계	권장 pair 수	목적
Smoke test	100-300	스키마·Unicode·token decode 검증
Pilot	약 1,000	QC·정규화·형태소·tokenizer pipeline 고정
최소 본 분석	20,000-50,000	안정적 EDA 및 주 추론
권장 본 분석	100,000+	도메인/길이 strata 및 이질성 분석
수동 품질 audit	300-500	병렬성·오류 유형·도메인 label 검증
Serving experiment	1,000-5,000	Track B 층화 표본

대규모 표본에서는 p-value가 거의 모든 작은 차이를 유의하게 만들 수 있으므로, 본 연구는 통계적 유의정보보다 effect size, interval estimate, hold-out performance, 증분 설명력을 우선한다.

9.2 층화 변수

- domain: general, administration, legal, news, technology, education, dialogue, other
- sentence_type: declarative, interrogative, imperative, title, list_item, short_phrase, other
- translation_direction: KO_TO_EN, EN_TO_KO, HUMAN_PARALLEL_UNKNOWN, UNKNOWN
- source_id: 원천 dataset/collection
- length_stratum: 영어 word/code point 또는 pair mean byte 기준 분위수

9.3 데이터 source 계층

가능한 경우 다음 우선순위를 따른다.

1. **Tier A — curated/official parallel corpus:** 명시적 번역 pair, 안정적 license와 metadata
2. **Tier B — benchmark parallel corpus:** 연구 benchmark 성격, 규모는 작더라도 품질 우수
3. **Tier C — web-mined parallel corpus:** 대규모 확보용, semantic QC를 강화

분석에서는 source tier를 저장하고 Tier A/B만으로 민감도 분석을 수행한다.

10. Pair Quality Control

10.1 Hard exclusion

다음 pair는 본 분석에서 제외한다.

- KO 또는 EN이 empty/null
- 완전 중복 pair
- 언어 식별이 명백히 잘못된 pair
- HTML/code/URL/table markup이 텍스트 대부분을 차지하는 pair
- zero-width/control character가 비정상적으로 과다한 pair
- decoder 오류 또는 tokenizer round-trip 검증 실패
- 자동 또는 수동 audit에서 의미 대응이 명백히 실패한 pair

10.2 Soft flags

삭제하지 않고 flag를 생성한다.

- short_text_flag
- long_text_flag
- high_digit_ratio_flag
- high_punctuation_ratio_flag
- script_mix_flag
- unicode_anomaly_flag
- translation_quality_review_flag
- named_entity_heavy_flag

10.3 의미 대응 QC

권장 3단계:

1. source-level metadata에서 병렬쌍 여부 확인
2. multilingual semantic similarity 또는 alignment score를 보조 QC로 계산
3. strata 기반 300-500 pair 수동 audit

수동 audit rubric:

점수	정의
2	의미·정보량이 실질적으로 동등
1	핵심 의미는 대응하나 경미한 누락·추가·의역 존재
0	다른 의미, 심각한 누락, 정렬 오류

Primary cohort는 2에 대응하는 자동/수동 기준을 목표로 하고, score 1을 포함한 결과를 sensitivity analysis로 분리할 수 있다.

Threat T-01 — QC model bias. 자동 semantic similarity 모델 자체도 언어별 tokenizer/representation bias를 가질 수 있다. 따라서 자동 QC score를 주 결과변수 생성에 사용하지 않고, curated-source subset과 수동 audit로 결과의 방향을 검증한다.

11. 정규화 규약

원문은 절대 덮어쓰지 않는다.

11.1 저장 필드

필드	정의
*_text_raw	source에서 받은 원문
*_text_nfc	Unicode NFC만 적용한 문자열
*_text_analysis	NFC + BOM 제거 + 외곽 whitespace trim; 내부 whitespace는 보존
normalization_rule_version	정규화 규칙 버전
normalization_ops	실제 적용 operation list
unicode_anomaly_flag	제어문자·zero-width·비정상 결합문자 등

11.2 금지되는 무음 정규화

Primary analysis text에는 다음을 수행하지 않는다.

- 내부 whitespace collapse
- punctuation ASCII 치환
- lowercasing
- full-width/half-width 통합
- NFKC 변환
- 숫자 치환
- emoji 제거
- 조사/어미 분리

NFKC, whitespace collapse, raw text 기준 결과는 별도 sensitivity track에서만 허용한다.

12. 데이터 스키마

12.1 D-01 Pair Registry

변수	설명
pair_id	문장쌍 고유 ID
source_id	원천 dataset ID
source_tier	A/B/C
domain	도메인
sentence_type	문장 유형
translation_direction	번역 방향
ko_text_raw, en_text_raw	원문
ko_text_nfc, en_text_nfc	NFC
ko_text_analysis, en_text_analysis	primary analysis text
pair_quality_status	accepted/review/rejected
pair_quality_score	optional alignment score
pair_version	pair registry 버전
source_license_note	license/usage note

12.2 D-02 Representation Features

변수군	주요 변수
Unicode length	codepoint_count, grapheme_count
lexical length	ko_eojeol_count, en_word_count
byte	utf8_bytes, bytes_per_codepoint, bytes_per_grapheme

변수군	주요 변수
whitespace	whitespace_count, whitespace_density, space_run_count
script	Hangul, Latin, digit, punctuation, symbol, other 비율
mixing	script_type_count, script_switch_count
special expression	URL/email/emoji/code-like pattern flags
morphology	morpheme/particle/ending/affix counts and ratios
provenance	feature extractor version, Unicode library/version

Definition DN-01 — char ambiguity 제거. char_count라는 모호한 명칭은 핵심 분석에서 사용하지 않는다. codepoint_count와 grapheme_count를 분리 저장한다. extended grapheme cluster는 Unicode \X 규칙을 따르는 구현을 사용하고 라이브러리 버전을 고정한다.

12.3 D-03 Morphology Measurement

변수	설명
morph_measurement_id	형태소 분석 실행 ID
pair_id	pair key
analyzer_name	예: Kiwi
analyzer_package_version	예: kiwipiepy package version
analyzer_model_version	model package/revision
analyzer_config_hash	설정 canonical hash
morpheme_sequence	form/POS sequence
morpheme_count	형태소 수
particle_count	조사 계열
ending_count	어미 계열
deriv_affix_count	파생 접사 계열
analysis_warning_flag	분석 실패/비정상 결과

POS feature mapping 원칙

Kiwi의 Sejong-derived tag 체계를 사용할 경우 사전 등록 예시는 다음과 같다.

- Particle: J*
- Ending: E*
- Derivational affix: XSN, XSV, XSA
- Primary function morpheme set: J* ∪ E*
- Extended sensitivity set: J* ∪ E* ∪ {XSN, XSV, XSA}

FunctionMorphemeRatio와 그 구성 요소인 ParticleRatio, EndingRatio를 같은 primary regression에 동시에 넣지 않는다.

12.4 D-04 Token Measurement

변수	설명
measurement_id	tokenizer 측정 ID
pair_id	pair key

변수	설명
tokenizer_id	o200k_base
tiktoken_version	실행 package version
encoding_file_sha256	raw encoding artifact hash
mergeable_ranks_hash	canonical rank mapping hash
pat_str_sha256	regex pattern hash
special_tokens_hash	canonical special-token mapping hash
ko_token_ids, en_token_ids	token ID array
ko_token_count, en_token_count	token 수
token_premium	TP_i
log_token_premium	$\log TP_i$
token_difference	ΔT_i
compression_penalty	pair-level token-per-byte ratio
roundtrip_ok	decode(encode(text)) 검증

12.5 D-05 Regex Chunk Measurement

변수	설명
chunk_measurement_id	측정 ID
pair_id	pair key
tokenizer_measurement_id	D-04 FK
ko_chunk_count, en_chunk_count	regex chunk 수
mean_chunk_bytes_*	평균 chunk byte 길이
p50_chunk_bytes_*, p90_chunk_bytes_*	chunk 길이 분포
tokens_per_chunk_*	chunk당 최종 token 수
chunk_type_share_*	letter/number/punctuation/whitespace 등

12.6 D-06 Model Run

변수	설명
run_id	serving 실행 ID
pair_id	pair key
model_id, model_revision	모델 및 revision
deployment_mode	local/provider/API-compatible
provider_id	provider가 있으면 기록
native_tokenizer_id	gpt-oss는 o200k_harmony 계열
prompt_template_version	semantic task template
chat_template_version	Harmony serialization version/hash
reasoning_effort	low/medium/high
temperature, top_p, max_new_tokens, seed	decode 설정
request_tokens	serialized input tokens
analysis_tokens	reasoning/analysis channel token, 가능 시
final_tokens	final channel token
total_output_tokens	전체 생성 token
ttft_ms	time to first token
latency_ms	end-to-end latency
decode_tokens_per_sec	decode throughput
finish_reason	종료 사유
cache_state	warm/cold/cache hit 가능 시

12.7 D-07 Cost / Compute Snapshot

변수	설명
cost_snapshot_id	snapshot ID
provider_id	provider 또는 LOCAL
model_id	model
effective_at	가격/전력단가 기준 시점
billing_basis	per-token / per-second / GPU-hour / local-energy
input_price_per_million	해당 시 존재할 경우
output_price_per_million	해당 시 존재할 경우
gpu_hour_price	provider가 GPU-hour 기준이면
energy_price_per_kwh	로컬 전력비 분석 시
source_reference	근거 source
estimated_cost	run-level 추정 비용

Decision D-02. gpt-oss는 open-weight 모델이므로 “모델 자체의 고정 API token price”를 가정하지 않는다. 비용은 deployment/provider 조건에 귀속시킨다. 로컬 실행에서는 가격 대신 latency, throughput, VRAM/RAM, 가능하면 energy proxy를 주 성능지표로 둔다.

13. 핵심 feature 정의

13.1 Token Premium

$$TP_i = \frac{T_{i,KO}}{T_{i,EN}}$$

$$\log TP_i = \log T_{i,KO} - \log T_{i,EN}$$

13.2 절대 Token 차이

$$\Delta T_i = T_{i,KO} - T_{i,EN}$$

13.3 Byte Density

$$\text{BytesPerCodePoint}_{i,l} = \frac{B_{i,l}}{C_{i,l}}$$

13.4 Whitespace Density

$$\text{WhitespaceDensity}_{i,l} = \frac{W_{i,l}}{C_{i,l}}$$

13.5 Pair Length Ratio

Primary surface length ratio:

$$\text{LengthRatio}_i = \log \left(\frac{C_{i,KO}}{C_{i,EN}} \right)$$

모든 accepted pair는 비어 있지 않으므로 primary 식에 임의의 +1 smoothing을 넣지 않는다. zero count가 가능한 특정 feature ratio만 사전 정의된 small constant 또는 count model을 사용한다.

13.6 Morpheme Density

$$\text{MorphemeDensity}_{i,KO} = \frac{\text{MorphemeCount}_{i,KO}}{\text{EojeolCount}_{i,KO}}$$

13.7 Particle / Ending Ratio

$$\text{ParticleRatio}_{i,KO} = \frac{\text{ParticleCount}_{i,KO}}{\text{MorphemeCount}_{i,KO}}$$

$$\text{EndingRatio}_{i,KO} = \frac{\text{EndingCount}_{i,KO}}{\text{MorphemeCount}_{i,KO}}$$

14. Tokenizer 고정 규약

14.1 Primary tokenizer

- Encoding: o200k_base
- Primary implementation: tiktoken
- Design-freeze reference version: tiktoken==0.13.0
- 실제 분석 시작 시 environment lockfile에 정확한 wheel/source hash까지 기록

공식 open-source 구현에서 o200k_base는 공개된 regex pattern(pat_str)과 mergeable rank table을 사용한다. 따라서 본 연구의 chunk feature는 해당 구현을 고정하면 재현 가능하다.

14.2 측정 원칙

1. Track A에는 chat template나 special token을 넣지 않는다.
2. text_analysis 문자열만 encode한다.
3. encode -> decode round-trip을 검증한다.
4. token ID뿐 아니라 decode_single_token_bytes 기반 token byte representation도 audit sample에 저장할 수 있다.
5. encoding file, regex, merge rank, special-token mapping을 각각 hash한다.
6. 라이브러리 버전만 같다는 이유로 tokenizer가 같다고 가정하지 않는다.

14.3 o200k_harmony와의 관계

gpt-oss serving track에서는 o200k_harmony와 Harmony message format을 사용한다. 공개 구현에서 o200k_harmony는 o200k_base의 regex pattern과 mergeable ranks를 재사용하고 Harmony용 special token을 추가한다. 따라서 **일반 raw text만 encode할 때와 실제 chat message를 Harmony serialization한 뒤 encode할 때를 구분해야 한다.**

Decision D-03. Track A는 o200k_base raw text efficiency를 측정한다. Track B는 o200k_harmony + Harmony serialization의 실제 request accounting을 측정한다. 두 결과는 같은 컬럼으로 합치지 않는다.

15. 형태소 분석 고정 규약

15.1 Primary analyzer

본 연구는 대규모 Python pipeline과 명시적인 POS sequence 저장이 필요한 점을 고려해 Kiwi Python binding(kiwipiepy)을 primary morphology analyzer 후보로 고정한다.

Design-freeze 시점 기준 문서 버전은 kiwipiepy 0.23.2이며, 실제 실행 시 다음을 모두 저장한다.

- package version
- model package/revision
- configuration
- user dictionary 사용 여부
- analyzer output schema version
- canonical config hash

15.2 분석 금지 사항

- 형태소 분석 결과를 tokenizer input으로 재조합하지 않는다.
- analyzer가 교정한 문자열을 Track A tokenizer 입력으로 사용하지 않는다.
- custom dictionary를 도메인별로 임의 추가한 뒤 다른 source와 비교하지 않는다.

Custom dictionary가 필요하면 모든 dataset에 동일 사전을 적용한 secondary track으로 분리한다.

15.3 형태소 feature block

Primary block:

- morpheme_density
- particle_ratio
- ending_ratio
- deriv_affix_ratio

Alternative block:

- morpheme_density
- function_morpheme_ratio
- deriv_affix_ratio

두 block을 병렬 비교해 구성비 변수의 공선성 영향을 점검한다.

16. 기초 분석 계획

16.1 핵심 분포

- KO/EN token count mean, median, SD, IQR, quantiles
- TP, logTP, ΔT
- CodePointRatio, ByteDensityRatio, CompressionPenalty
- tokens/codepoint, tokens/byte
- morphology feature distributions
- regex chunk feature distributions

16.2 필수 시각화

- F01 KO vs EN paired token scatter, identity line
- F02 TP histogram + ECDF
- F03 domain별 TP violin/box
- F04 exact decomposition component distribution
- F05 ByteDensityRatio vs CompressionPenalty
- F06 morphology density vs logTP partial relationship
- F07 extreme TP case audit panel
- F08 source/domain forest plot
- F09 Track B request/output/latency comparison

16.3 Extreme-case audit

최상위/최하위 TP 각 50-100건을 자동 추출하고 다음을 한 화면에서 확인한다.

- raw / normalized KO-EN text
- codepoints / graphemes / bytes
- regex chunks
- token IDs and decoded token bytes
- morphology sequence
- domain/source/direction
- anomaly flags

극단값은 먼저 삭제하지 않는다. 원인을 분류한 후 hard error만 제외하고 valid extreme은 분석에 유지한다.

17. RQ1 통계 추론

17.1 Primary test

logTP에 대한 one-sample signed-rank inference:

$$H_0 : \text{Median}(\log TP) = 0$$

$$H_1 : \text{Median}(\log TP) > 0$$

분포의 대칭성 가정이 의심되면 sign test를 robustness check로 병행한다.

17.2 Confidence intervals

주 통계량에 대해 pair-level bootstrap 95% CI를 산출한다.

- median TP
- geometric mean TP
- median logTP
- $P(TP > 1)$
- median absolute token difference

source 불균형이 큰 경우 source-stratified bootstrap을 기본으로 사용하고, source cluster bootstrap은 source level 수가 충분할 때 보조로 수행한다.

17.3 보고 원칙

- p-value 단독 보고 금지
- effect size + CI 우선
- 표본이 매우 큰 경우 “statistically significant” 보다 실제 premium 크기를 해석

18. 메인 설명모형

18.1 Outcome A — Total Premium

$$Y_i^{(A)} = \log TP_i$$

기본 형태:

$$\begin{aligned}
Y_i^{(A)} = & \beta_0 + \beta_1 \log \text{CodePointRatio}_i + \beta_2 \log \text{ByteDensityRatio}_i \\
& + \beta_3 \Delta \text{WhitespaceDensity}_i + \beta_4^\top \text{ScriptFeatures}_i \\
& + \beta_5^\top \text{Morphology}_i + \gamma^\top \text{Domain}_i + \delta^\top \text{SentenceType}_i \\
& + \eta^\top \text{TranslationDirection}_i + u_{\text{source}[i]} + \epsilon_i
\end{aligned}$$

18.2 Outcome B — Compression Penalty

$$Y_i^{(B)} = \log \text{CompressionPenalty}_i$$

이 outcome은 UTF-8 byte 량의 총 차이를 분리한 뒤 tokenizer-specific fragmentation/compression 차이를 설명하는 보조 메커니즘 outcome이다.

예시:

$$\begin{aligned}
Y_i^{(B)} = & \alpha_0 + \alpha_1 \Delta \text{WhitespaceDensity}_i + \alpha_2^\top \text{ScriptFeatures}_i \\
& + \alpha_3^\top \text{Morphology}_i + \lambda^\top \text{Domain}_i + u_{\text{source}[i]} + \nu_i
\end{aligned}$$

Interpretation Rule IR-02. logTP 모형은 전체 premium을 설명하고, logCompressionPenalty 모형은 byte 표현량을 제외한 tokenizer compression 비대칭을 설명한다. 두 outcome을 구분해 보고한다.

19. 단계적 모형 비교

19.1 Model blocks

$$\mathcal{M}_0 : Y \sim \text{length} + \text{domain} + \text{sentence type} + \text{translation direction} + \text{source}$$

$$\mathcal{M}_1 : Y \sim \mathcal{M}_0 + \text{byte/whitespace/script/surface features}$$

$$\mathcal{M}_2 : Y \sim \mathcal{M}_1 + \text{morphology block}$$

$$\mathcal{M}_3 : Y \sim \mathcal{M}_2 + \text{regex-chunk/tokenizer mechanism features}$$

19.2 핵심 질문

M2 - M1에서 형태소 feature block이 표면형·도메인·출처를 통제한 뒤에도 실질적인 증분 설명력을 제공하는가?

평가:

- likelihood-ratio test, 적용 가능한 nested model에서만
- AIC / BIC
- marginal/conditional R^2 for mixed models
- adjusted R^2 for fixed OLS variants
- out-of-sample MAE / RMSE / R^2
- morphology block partial R^2 또는 likelihood-based effect

19.3 M3의 역할

M3는 tokenizer 구조를 설명하기 위한 mechanism audit다. regex chunk feature는 outcome의 생성 과정에 매우 가까우므로, M3가 높은 R^2 를 보이는 것은 예상 가능한 결과다. M3의 목적은 predictive win이 아니라 어떤 **surface/morphological patterns**가 **chunk fragmentation**과 연결되는지 추적하는 것이다.

20. Source effect와 식별 가능성

20.1 Random vs Fixed

- source level 수가 충분하고 각 source에 충분한 관측치가 있으며 random-effects 가정이 합리적이면 random intercept 사용
- source 수가 적거나 특정 source가 특정 domain과 거의 완전히 결합되어 있으면 source fixed effect 또는 source-domain composite 사용
- domain과 source가 완전 공선이면 둘의 독립계수를 동시에 해석하지 않는다

20.2 Identifiability Gate

본 회귀 실행 전 다음을 확인한다.

- source $\times$ domain contingency table
- source $\times$ translation_direction table
- condition number
- VIF 또는 generalized VIF
- zero/near-zero variance features
- pairwise Spearman/Pearson correlation

Gate G-ID. domain과 source가 구조적으로 식별 불가능하면 coefficient table을 강제로 산출하지 않고 설계를 재매핑한다.

21. 공선성 및 compositional feature 처리

조사 비율, 어미 비율, 기능형태소 비율은 서로 부분적으로 중첩된다. Hangul/Latin/digit/punctuation 비율 또한 합이 1에 가까운 composition이다.

원칙:

- function ratio와 그 하위 component를 동일 primary model에 중복 투입하지 않는다.
- script share는 하나의 reference category를 제외하거나 적절한 compositional transform을 sensitivity로 사용한다.
- VIF만으로 feature를 자동 삭제하지 않고 feature semantics를 먼저 검토한다.
- Elastic Net feature selection 결과를 설명모형의 “진실”로 대체하지 않는다.

22. 예측모형

목적	권장 모델	역할
설명·가설검정	Mixed-effects / fixed-effects regression	coefficient + CI
공선성 대응	Elastic Net	stable predictive feature set
비선형 예측	CatBoost 또는 XGBoost	nonlinearities / interactions
이상치 강건성	Huber regression	heavy-tail sensitivity
상위 premium 분석	Quantile regression	high-TP tail

SHAP 및 permutation importance는 예측 기여도로만 해석한다.

23. 데이터 분할과 일반화 검증

23.1 설명모형

설명모형의 계수 추론은 전체 analysis cohort에서 수행하되 bootstrap 및 model diagnostics를 적용한다.

23.2 예측모형

- final hold-out: 15-20%
- tuning: training partition 내부에서만
- source 수가 충분하면 GroupKFold(source_id)
- source 수가 적고 domain과 결합되면 pair-level split + near-duplicate cluster group을 primary로 하고 source-held-out test를 secondary OOD test로 분리
- continuous target의 stratification이 필요하면 training distribution에서 생성한 target quantile bins를 fold balancing 용도로만 사용

Leakage Rule LR-01. 같은 원문에서 파생된 paraphrase/duplicate cluster가 train/test 양쪽에 들어가지 않도록 text hash 및 near-duplicate cluster를 group key로 활용한다.

24. Robustness / Sensitivity Analysis

최소 다음을 수행한다.

1. raw text vs analysis-normalized text
2. NFC-only vs primary analysis text
3. full accepted cohort vs curated-source-only cohort
4. anomaly flag 포함 vs 제외
5. extreme length 상·하위 strata 제외
6. translation direction known-only subset
7. morphology primary block vs alternative function-morpheme block
8. OLS/mixed model vs Huber/quantile regression
9. source fixed vs random specification 가능 범위
10. sign test vs signed-rank test for RQ1

다중 비교 보정은 secondary feature-level 및 interaction-level hypothesis family에 Benjamini-Hochberg FDR을 적용한다. 단일 primary RQ1에는 불필요한 FDR 보정을 적용하지 않는다.

25. Track A — Tokenizer Intrinsic Analysis

25.1 목적

o200k_base가 동일 의미의 KO-EN raw text를 얼마나 다르게 segment/compress하는지 측정한다.

25.2 입력

- *_text_analysis
- chat template 없음
- system/user role 없음
- special token 없음

25.3 산출

- token IDs
- token count
- tokens/codepoint
- tokens/grapheme
- tokens/byte
- exact TP decomposition
- regex chunk statistics

25.4 성공 조건

- 모든 accepted pair에 token count > 0
- encode/decode roundtrip 100% PASS
- tokenizer artifact hashes 저장
- pilot 100건 token audit에서 script/byte decoding 이상 없음

26. Track B — gpt-oss Serving Experiment

26.1 목적

실제 chat serialization과 generation에서 언어에 따른 serving overhead를 측정한다. 이 실험은 Track A의 tokenizer intrinsic result를 대체하지 않는다.

26.2 모델

- gpt-oss-20b
- gpt-oss-120b

두 모델은 Harmony response format과 o200k_harmony tokenizer 계열을 사용한다. 동일한 serialized message라면 모델 크기만으로 request token count가 달라질 것으로 기대하지 않는다. 차이는 주로 generation behavior, reasoning effort, latency, throughput, hardware/backend에서 발생할 수 있다.

26.3 세 가지 protocol

B1. Request-only accounting

목적: serialization overhead를 분리한다.

- 동일 semantic content
- 동일 role structure
- KO/EN template 구조 동등
- generation 없이 tokenizer/request count 계산 가능하면 우선 사용

B2. Controlled generation

목적: output verbosity confounding을 최소화한다.

예:

- fixed-schema JSON extraction
- short classification
- constrained answer slots

B3. Natural generation

목적: 실제 사용자 경험에서 language별 output length/latency 차이를 탐색한다.

- realistic instruction task
- 결과는 exploratory로 명시
- output semantic equivalence를 별도 QC

26.4 고정 변수

- Harmony chat template revision/hash
- system/developer/user message structure
- reasoning effort
- temperature
- top-p
- max output tokens
- seed, backend가 지원하면
- batch size
- quantization
- device/backend
- warm-up 횟수
- concurrency

26.5 latency 측정

단일 latency_ms만 보고하지 않는다.

- TTFT
- end-to-end latency
- decode tokens/sec
- prompt processing tokens/sec, 가능 시
- cold/warm run
- p50/p95 latency

27. Serving-level token ratio

모델 m , pair i :

$$R_{m,i}^{req} = \frac{I_{m,i,KO}}{I_{m,i,EN}}$$

출력:

$$R_{m,i}^{out} = \frac{O_{m,i,KO}}{O_{m,i,EN}}$$

전체:

$$R_{m,i}^{total} = \frac{I_{m,i,KO} + O_{m,i,KO}}{I_{m,i,EN} + O_{m,i,EN}}$$

reasoning/analysis channel이 별도로 관측되면 final channel과 분리한다.

Threat T-02 — output length confounding. 자연 생성에서 output token 차이는 tokenizer뿐 아니라 모델의 verbosity, reasoning path, language-specific style에 의해 달라진다. 따라서 B2 controlled generation과 B3 natural generation을 분리한다.

28. 비용 모형

28.1 Provider per-token billing

provider가 input/output token 가격을 제공하는 경우:

$$\text{Cost}_{m,i,l} = \frac{I_{m,i,l}p_{m,p}^{in} + O_{m,i,l}p_{m,p}^{out}}{10^6}$$

$$\text{CostRatio}_{m,i} = \frac{\text{Cost}_{m,i,KO}}{\text{Cost}_{m,i,EN}}$$

여기서 p 는 모델만이 아니라 provider p 와 effective date에 종속된다.

28.2 Local deployment

로컬 실행에서는 고정 per-token price를 임의 생성하지 않는다. 가능한 경우 다음을 보고한다.

- total runtime
- throughput
- peak VRAM/RAM
- 평균 GPU power draw
- energy use estimate

전력 기반 직접비 추정이 필요하다면:

$$\text{EnergyCost} = \text{kWh} \times \text{PricePerKWh}$$

hardware amortization은 별도의 시나리오 분석으로만 둔다.

29. 다중 검정과 통계 보고

- Primary RQ1: one primary test + CI
- Morphology block: block-level test 우선
- 개별 morphology coefficient: 95% CI + FDR adjusted q-value
- domain interaction family: 별도 FDR family
- post-hoc extreme-case exploration은 confirmatory result와 구분

모든 회귀표에는 다음을 기록한다.

- N pairs
- source count
- domain count
- estimator
- SE type
- random/fixed effects specification

- reference levels
- missingness handling
- R^2 또는 likelihood metric
- preprocessing/version ID

30. 재현성 규약

30.1 환경

필수 고정 항목:

- Python version
- OS/container info
- tiktoken version + artifact hashes
- kiwipiepy + model version/hash
- regex/Unicode version
- pandas/numpy/scipy/statsmodels/sklearn versions
- gradient boosting package version

30.2 데이터 lineage

각 release에 다음을 저장한다.

- raw file hash
- source registry snapshot
- transformation manifest
- rejected pair list + reason
- accepted pair hash
- feature table hash
- tokenizer measurement hash
- morphology measurement hash
- analysis config hash
- Git commit SHA

30.3 Seed policy

- global seed
- split seed
- bootstrap seed
- model tuning seed
- serving generation seed, backend 지원 시

seed가 determinism을 보장하지 않는 backend는 `determinism_status`로 별도 표시한다.

31. 연구 Gate

G0 — Design Freeze

PASS 조건:

- RQ/estimand 고정
- primary outcome 고정
- tokenizer/analyzer 고정
- QC rubric 고정

- main/sensitivity boundary 고정

G1 — Data Integrity

PASS 조건:

- pair IDs unique
- null/duplicate checks
- LID/QC pass rate 보고
- source/license metadata 완성

G2 — Representation Integrity

PASS 조건:

- Unicode normalization audit
- codepoint/grapheme/byte 계산 단위 테스트
- exact decomposition numerical identity 검증

수치적으로 모든 pair에서:

$$|\log TP_i - (\log CR_i + \log BDR_i + \log CP_i)| < \varepsilon$$

여기서 CR 은 code point ratio, BDR 은 byte density ratio, CP 는 compression penalty다.

G3 — Tokenizer Integrity

PASS 조건:

- roundtrip 100%
- tiktoken version/hash 기록
- pat_str hash 기록
- audit sample token bytes inspection

G4 — Morphology Integrity

PASS 조건:

- analyzer/version/hash 기록
- failure rate 보고
- POS mapping table freeze
- sample manual inspection

G5 — Analysis Readiness

PASS 조건:

- identifiability check
- collinearity report
- analysis cohort freeze
- split manifest freeze

G6 — Release

PASS 조건:

- 모든 표/그림 자동 생성
- notebook/script 재실행 가능

- 결과 manifest 생성
- final report에 limitation 포함

32. 핵심 위협과 해석 제한

T-03 Translationese

번역 방향에 따라 표현 길이와 어순이 달라질 수 있다. 해결: direction stratification/interaction 및 known-direction subset.

T-04 Domain-source confounding

특정 source가 특정 domain을 독점하면 domain effect와 source effect가 분리되지 않는다. 해결: identifiability gate와 source-domain composite sensitivity.

T-05 Analyzer measurement error

형태소 analyzer가 domain, 신조어, 고유명사에서 오류를 낼 수 있다. 해결: analyzer version freeze, sample audit, curated subset, warning feature.

T-06 Deterministic feature overlap

byte density, code point length, token-per-byte는 서로 수학적으로 연결된다. 해결: exact decomposition을 descriptive accounting으로 우선하고, 회귀에는 동일 항등식의 구성요소를 무비판적으로 중복 투입하지 않는다.

T-07 Tokenizer-specific generalization

o200k_base 결과는 모든 tokenizer에 자동 일반화되지 않는다. 결론 문구는 “o200k_base 기준”을 명시한다.

T-08 Cost generalization

가격은 시간·provider·deployment에 따라 바뀐다. 비용은 snapshot으로만 보고한다.

33. 결과 해석 프레임

가능한 결과와 해석은 다음처럼 사전 정의한다.

Case A — $TP > 1$, ByteDensity가 주로 설명

한국어 premium의 상당 부분이 UTF-8 representation load와 표현 길이 차이에서 기인하고, byte-normalized compression penalty는 상대적으로 작다는 해석이 가능하다.

Case B — $TP > 1$, CompressionPenalty도 큼

같은 byte량을 기준으로 해도 tokenizer가 한국어에서 더 많은 token을 산출하는 구조적 비대칭이 남는다는 의미다.

Case C — Morphology block이 M1 대비 실질적으로 개선

형태소 구조 feature가 단순 길이·byte·공백·script composition 이상으로 token premium variation과 연관된다는 증거다. 인과효과라고 표현하지 않는다.

Case D — Morphology block 증분이 거의 없음

형태소 자체가 “무관하다”기보다, 현재 analyzer/feature 정의에서 표면형 구조를 넘어서는 추가 예측·설명 정보가 제한적이라는 결론이 적절하다.

Case E — Domain interaction이 큼

단일 평균 premium보다 domain-specific tokenizer efficiency가 실무적으로 중요할 수 있다.

34. Traceability Matrix

RQ	Primary data	Estimand / outcome	Method	핵심 산출물
RQ1	D-01 + D-04	median logTP	signed-rank + bootstrap	T01, F01-F03
RQ2	D-02 + D-04	exact decomposition	identity + distribution	T02, F04-F05
RQ3	D-02 + D-04	logTP / logCompressionPenalty	M1 regression	T03, F06
RQ4	D-03 + D-04	incremental morphology value	M2 vs M1	T04, coefficient plot
RQ5	D-05 + D-04	chunk/token mechanism	M3 mechanism audit	T05, extreme audit
RQ6	D-01..D-05	strata-specific effects	interactions / forest plot	T06, F08
RQ7	D-06 + D-07	request/output/latency/cost ratios	paired serving analysis	T07, F09

35. 최종 표·그림 설계

Tables

- **T01** Sample composition and QC flow
- **T02** Overall KO-EN tokenization statistics
- **T03** Exact premium decomposition summary
- **T04** Main explanatory models M0-M2
- **T05** Mechanism audit M3
- **T06** Domain/source heterogeneity
- **T07** Serving-level experiment
- **T08** Robustness summary

Figures

- **F01** KO vs EN token count scatter
- **F02** TP distribution + ECDF
- **F03** Domain-specific TP
- **F04** Log decomposition components
- **F05** UTF-8 load vs tokenizer compression penalty
- **F06** Partial effects of morphology features
- **F07** Token/chunk/morpheme case audit
- **F08** Domain/source forest plot
- **F09** gpt-oss serving comparison

36. 프로젝트 정보구조(IA)

권장 repository 구조:

```

koen-token-premium/
├── README.md
├── pyproject.toml
├── uv.lock / requirements.lock
├── .env.example
├── configs/
│   ├── research_v1.yaml
│   ├── normalization_v1.yaml
│   ├── morphology_v1.yaml
│   ├── tokenizer_v1.yaml
│   └── serving_v1.yaml
├── docs/
│   ├── 00_RESEARCH_SPEC_v1.0.md
│   ├── 01_DATA_DICTIONARY.md
│   ├── 02_QC_PROTOCOL.md
│   ├── 03_ANALYSIS_PLAN.md
│   ├── 04_SERVING_PROTOCOL.md
│   └── CHANGELOG.md
├── data/
│   ├── registry/
│   ├── raw/                # immutable
│   ├── interim/
│   ├── processed/
│   └── releases/
├── notebooks/
│   ├── 00_environment_repro.ipynb
│   ├── 01_build_pair_registry.ipynb
│   ├── 02_normalize_and_qc.ipynb
│   ├── 03_representation_features.ipynb
│   ├── 04_morphology_features.ipynb
│   ├── 05_o200k_measurement.ipynb
│   ├── 06_regex_chunk_audit.ipynb
│   ├── 07_eda_and_decomposition.ipynb
│   ├── 08_primary_inference.ipynb
│   ├── 09_explanatory_models.ipynb
│   ├── 10_predictive_models.ipynb
│   ├── 11_robustness.ipynb
│   ├── 12_gpt_oss_serving.ipynb
│   └── 13_release_tables_figures.ipynb
├── src/
│   └── koen_tp/
│       ├── io.py
│       ├── normalization.py
│       ├── qc.py
│       ├── unicode_features.py
│       └── morphology.py

```

```

|       |—— tokenization.py
|       |—— chunking.py
|       |—— decomposition.py
|       |—— statistics.py
|       |—— modeling.py
|       |—— serving.py
|—— tests/
|   |—— test_normalization.py
|   |—— test_unicode_features.py
|   |—— test_tokenizer_roundtrip.py
|   |—— test_decomposition_identity.py
|   |—— test_schema.py
|—— outputs/
|   |—— manifests/
|   |—— tables/
|   |—— figures/
|   |—— models/
|   |—— reports/

```

37. Notebook 실행 순서

Phase 0 — Environment

00_environment_repro.ipynb

- package versions
- CPU/GPU/OS
- tokenizer/analyzer hash
- Unicode/regex version

Phase 1 — Data Contract

01_build_pair_registry.ipynb

- source ingest
- stable pair ID
- metadata schema
- raw hash

Phase 2 — Normalization & QC

02_normalize_and_qc.ipynb

- NFC/analysis text
- anomaly flags
- duplicate/LID/parallel quality
- QC flow table

Phase 3 — Feature Layer

03_representation_features.ipynb

- codepoint/grapheme/byte/whitespace/script

04_morphology_features.ipynb

- Kiwi sequence/POS
- feature mapping

Phase 4 — Tokenizer Measurement

05_o200k_measurement.ipynb

- token IDs/counts
- roundtrip
- TP
- exact decomposition

06_regex_chunk_audit.ipynb

- fixed regex chunks
- chunk features

Phase 5 — Statistics

07_eda_and_decomposition.ipynb

- descriptive tables/figures

08_primary_inference.ipynb

- RQ1 test + bootstrap

09_explanatory_models.ipynb

- M0-M3

10_predictive_models.ipynb

- Elastic Net / boosting

11_robustness.ipynb

- sensitivity matrix

Phase 6 — Serving

12_gpt_oss_serving.ipynb

- B1/B2/B3 protocols

Phase 7 — Release

13_release_tables_figures.ipynb

- table/figure manifest
- report artifacts
- reproducibility bundle

38. 파일 명명 규칙

예시:

PAIR_REGISTRY_v001.parquet
REP_FEATURES_v001.parquet

MORPH_FEATURES_KIWI_v001.parquet
 TOKEN_O200K_BASE_v001.parquet
 CHUNK_O200K_BASE_v001.parquet
 MODEL_RUN_GPT_OSS_v001.parquet
 ANALYSIS_RELEASE_20260816_v001.json

모든 release manifest에는 SHA-256, row count, schema version, code commit을 포함한다.

39. 분석 전 체크리스트

- ☐ Research questions와 estimand freeze
- ☐ Primary tokenizer와 implementation freeze
- ☐ Morphology analyzer/POS mapping freeze
- ☐ Source registry와 license note 완료
- ☐ Pair QC rubric freeze
- ☐ Unicode normalization tests PASS
- ☐ Tokenizer roundtrip tests PASS
- ☐ Exact decomposition identity tests PASS
- ☐ Morphology sample audit PASS
- ☐ Source-domain identifiability audit PASS
- ☐ Analysis cohort hash freeze
- ☐ Train/hold-out manifest freeze
- ☐ Figure/table IDs freeze

40. 최종 논문/보고서 서술용 핵심 문단

본 연구는 한국어-영어 의미 대응 문장쌍을 대상으로 특정 tokenizer에서 발생하는 Tokenization Premium을 측정하고, 이를 표현 길이, UTF-8 byte density, tokenizer compression의 세 구성요소로 정확히 분해한다. Tokenization Premium은 한국어와 영어의 최종 token count 비율로 정의하며, 동일 의미 pair 안에서 비교함으로써 topic과 의도 차이를 최대한 통제한다. 또한 공백, 문자군, 숫자·구두점, script mixing 등 표면형 feature와 한국어 형태소 밀도, 조사·어미 비율 등의 형태소 feature를 추출하여 premium의 조건부 변이를 분석한다.

본 연구는 형태소 분석과 tokenizer의 pre-token stage를 명시적으로 구분한다. 형태소 분석은 한국어 문장의 문법적·형태적 구조를 기술하기 위한 외부 분석이며, tokenizer의 실제 token 경계를 규정하지 않는다. o200k_base에 대해서는 고정된 tiktoken 구현에서 공개된 regex chunking과 mergeable rank를 재현하여 tokenizer mechanism feature를 별도로 측정한다. 형태소 feature의 증분 설명력은 surface representation model에 추가하는 방식으로 평가하고, tokenizer-derived chunk feature는 결과 생성 과정에 가까운 mechanism audit로 분리한다.

주 추론은 pair-level log Tokenization Premium을 대상으로 수행한다. 전체 premium의 존재 여부는 logTP의 중앙값이 0보다 큰지 검정하고 bootstrap confidence interval을 함께 제시한다. 설명모형에서는 길이, byte, whitespace, script, domain, sentence type, translation direction 및 source를 통제된 뒤 형태소 feature block의 추가 설명력을 평가한다. 결과는 인과효과가 아니라 특정 tokenizer와 데이터 범위에서의 조건부 통계적 연관성으로 해석한다.

Serving-level 보조 실험은 gpt-oss-20b와 gpt-oss-120b를 대상으로 Harmony serialization 조건에서 request token, reasoning/analysis token, final output token, TTFT, end-to-end latency 및 throughput을 측정한다. 두 모델의 native format/tokenizer가 공유되는 조건에서는 모델 크기 자체가 동일 serialized input의 token count를 바꾸는 요인으로 해석되지 않으며, serving 결과는 generation behavior와 deployment 조건의 차이를 포함하는 별도 계층으로 보고한다.

41. 참고문헌 및 구현 기준 자료

1. Petrov, A., La Malfa, E., Torr, P. H. S., & Bibi, A. (2023). Language Model Tokenizers Introduce Unfairness Between Languages. arXiv:2305.15425.
2. Ahia, O., Kumar, S., Gonen, H., Kasai, J., Mortensen, D. R., Smith, N. A., & Tsvetkov, Y. (2023). Do All Languages Cost the Same? Tokenization in the Era of Commercial Language Models. EMNLP 2023, 9904-9923.
3. Velayuthan, M., & Sarveswaran, K. (2025). Egalitarian Language Representation in Language Models: It All Begins with Tokenizers. COLING 2025.
4. Lee, J. et al. (2024). Length-aware Byte Pair Encoding for Mitigating Over-segmentation in Korean. Findings of ACL 2024.
5. Lee, D. H. et al. (2026). Morpheme-Based Subword Tokenization for Korean Language Models. EACL 2026 Short Papers.
6. Roy, A., Roy, P., & Patel, H. (2026). Measuring the Tokenization Premium: A Cost Audit for Underserved Language Communities. arXiv:2608.09046.
7. OpenAI. tiktoken open-source implementation, o200k_base / o200k_harmony constructors. Design-freeze reference: tiktoken 0.13.0.
8. OpenAI. (2025). Introducing gpt-oss and OpenAI Harmony Response Format.
9. Kiwi / kiwipiepy project documentation. Design-freeze documentation observed at kiwipiepy 0.23.2.

Appendix A. 결정 로그

ID	결정	이유
D-01	RQ1 primary test를 logTP로 변경	ratio hypothesis와 검정대상 일치
D-02	비용을 provider/deployment snapshot으로 일반화	gpt-oss open-weight 특성 반영
D-03	Track A o200k_base, Track B o200k_harmony 분리	raw-text token efficiency와 chat serialization 분리
D-04	exact decomposition을 연구 중	UTF-8 load와 tokenizer
D-05	심식으로 승격	compression을 구조적으로 분리
D-06	M3를 mechanism audit로 제한	tokenizer-derived feature의 post-treatment/기계적 성격 반영
D-07	char_count 대신 codepoint/grapheme 분리	Unicode 단위 모호성 제거
D-07	function morpheme와 하위 ratio 동시 투입 금지	구조적 공선성 방지

Appendix B. 최소 산출물 계약

최종 연구 release는 최소 다음을 포함한다.

1. research_spec.md
2. source_registry.csv/parquet
3. pair_registry.parquet
4. representation_features.parquet
5. morphology_features.parquet
6. token_measurements_o200k.parquet
7. chunk_measurements_o200k.parquet
8. analysis_manifest.json

9. tables/*.csv
10. figures/*.png|svg
11. notebooks/*.ipynb
12. environment lockfile
13. final_report.pdf

이 중 raw data는 license와 배포 조건에 따라 release bundle에서 제외할 수 있으며, 대신 source hash와 acquisition metadata를 남긴다.